```python
#exp2
def mp(x, w, t):
    return int(sum(a * b for a, b in zip(x, w)) >= t)

for a in (0, 1):
    for b in (0, 1):
        print(a, b,
              "AND:", mp([a, b], [1, 1], 2),
              "OR:", mp([a, b], [1, 1], 1),
              "XOR:", mp([mp([a, b], [1, -1], 1),
                          mp([a, b], [-1, 1], 1)], [1, 1], 1))
```

Output:

```
0 0 AND: 0 OR: 0 XOR: 0
0 1 AND: 0 OR: 1 XOR: 1
1 0 AND: 0 OR: 1 XOR: 1
1 1 AND: 1 OR: 1 XOR: 0
```

```python
#EXP3
X = [(1, 1), (1, -1), (-1, 1), (-1, -1)]
OR = [1, 1, 1, -1]

# Perceptron rule (OR)
w1 = w2 = b = 0
for epoch in range(1, 5):
    for (x1, x2), y in zip(X, OR):
        out = 1 if (w1 * x1 + w2 * x2 + b) >= 0 else -1
        if out != y:
            w1 += x1 * y
            w2 += x2 * y
            b += y

print(f"Final Weights:, w1={w1}, w2={w2}, b={b}")
```

Output:

```
Final Weights: w1=1, w2=1, b=1
```

```
#exp4
import numpy as np

X = np.array([[1, 0, 1, 0], [0, 1, 0, 1], [1, 1, 1, 1], [0,
0, 0, 0]])
W1 = np.random.rand(4, 2)  # encoder
W2 = np.random.rand(2, 4)  # decoder

for epoch in range(2000):
    H = X @ W1
    Y = H @ W2
    E = Y - X

    W1 -= 0.05 * X.T @ (E @ W2.T)
    W2 -= 0.05 * H.T @ E

print(np.round(Y))
```

```
Output:
[[ 1.  0.  1. -0.]
 [ 0.  1. -0.  1.]
 [ 1.  1.  1.  1.]
 [ 0.  0.  0.  0.]]
```

```
#exp 5
import numpy as np

A = np.array([0.2, 0.5, 0.8])
B = np.array([0.4, 0.7, 0.3])

print("Union:       ", np.maximum(A, B))
print("Intersection:", np.minimum(A, B))
print("Complement A:", 1 - A)

print("\nRelation (A x B):\n", np.minimum.outer(A, B))
```

```
Output:
Union:        [0.4 0.7 0.8]
Intersection: [0.2 0.5 0.3]
Complement A: [0.8 0.5 0.2]

Relation (A x B):
 [[0.2 0.2 0.2]
 [0.4 0.5 0.3]
 [0.4 0.7 0.3]]
```

```
#exp6
import numpy as np

x = np.array([1, 2, 3, 4, 5])
mu = np.array([0.1, 0.4, 0.8, 0.8, 0.3])

centroid = np.sum(x * mu) / np.sum(mu)

max = x[mu == np.max(mu)]

mom = np.mean(max)
som = np.min(max)
lom = np.max(max)

print("Centroid (COG)   :", round(centroid, 2))
print("Mean of Maxima   :", round(mom, 2))
print("Smallest of Maxima:", round(som, 2))
print("Largest of Maxima :", round(lom, 2))
```

OutPut:

```
Centroid (COG)   : 3.33
Mean of Maxima   : 3.5
Smallest of Maxima: 3
Largest of Maxima : 4
```

```
#exp7

def fuzzy_wash_time(dirt):
    low_dirt  = max(0, (5 - dirt) / 5)
    med_dirt  = max(0, 1 - abs(dirt - 5) / 5)
    high_dirt = max(0, (dirt - 5) / 5)

    time = (low_dirt * 0) + (med_dirt * 30) + (high_dirt *
60)
    total_weight = low_dirt + med_dirt + high_dirt  # Fixed
typo here

    return time / total_weight

dirt_level = 8
print(f"Wash Time for Dirt {dirt_level}:
{round(fuzzy_wash_time(dirt_level), 2)} mins”)
```

OutPut:

```
Wash Time for Dirt 8: 48.0 mins
```

```
#exp8
pop = [[1, 0, 1], [0, 1, 1], [1, 1, 1], [1, 0, 0]]

print("initial population:",pop)


pop.sort(reverse=True)
p1, p2 = pop[0], pop[1]
print("Parents:", p1, p2)

c1, c2 = p1[:1] + p2[1:], p2[:1] + p1[1:]
print("Children:", c1, c2)

c1[0] ^= 1
print("Mutated:", c1)
```

```
OutPut:
population:[[1, 0, 1], [0, 1, 1], [1, 1, 1], [1, 0,0]]
Parents: [1, 1, 1] [1, 0, 1]
Children: [1, 0, 1] [1, 1, 1]
Mutated: [0, 0, 1]
```